

Projeto prático

Simulador de escalonamento de tarefas

Sistemas Operacionais

1 Objetivo

Construir, em Python, um simulador dos algoritmos de escalonamento estudados em sala, capaz de reproduzir numericamente os cenários das aulas e de expor o fenômeno da inversão de prioridades e seu mecanismo de correção.

A arquitetura do programa é livre. Não há estrutura de pastas obrigatória, nomes de classes exigidos nem interface imposta: linha de comando, janela gráfica ou caderno interativo são todos aceitáveis. O que é avaliado é o comportamento do simulador, e não a forma como ele foi organizado por dentro.

O que está em jogo

Um simulador de escalonamento é um exercício de separação entre **política** e **mecanismo**. O mecanismo, que avança o relógio, mantém a fila de prontas e cobra a troca de contexto, é único. A política, que escolhe qual tarefa recebe o processador, muda seis vezes. Se essa separação estiver correta, cada algoritmo cabe em poucas linhas. Se não estiver, cada algoritmo vira um programa inteiro, e os seis divergem entre si.

2 Escopo

O simulador precisa cumprir os dez requisitos abaixo. Nada além disso é cobrado.

R1 a R4 correspondem à Aula 5 e R5 e R6 à Aula 6. R7 e R8 completam o tratamento de prioridades, R9 é o que permite generalizar as conclusões além dos cenários de estudo, e R10 trata da forma da entrega. Todos são obrigatórios.

R1. Os seis algoritmos de escalonamento

Sigla	Algoritmo	Preempção	Quantum	Critério de escolha
FCFS	First-Come, First-Served	não	não	menor instante de ingresso
SJF	Shortest Job First	não	não	menor tempo de processamento t_p
SRTF	Shortest Remaining Time First	sim	não	menor t_p restante
RR	Round-Robin	por quantum	sim	primeiro da fila circular
PRIOc	Prioridade cooperativa	não	não	maior prioridade
PRIOp	Prioridade preemptiva	sim	não	maior prioridade

R2. Entrada de tarefas

Uma tarefa é descrita por identificador, instante de **ingresso**, tempo de **processamento** t_p e prioridade. Quem executa o simulador escolhe **quantas tarefas** quer e, em seguida, **como** informá-las:

- **digitando** os dados de cada tarefa, campo por campo;
- deixando o simulador **sortear** tudo, conforme R9.

Nos dois casos, nada disso pode exigir a edição do código-fonte, e o mesmo vale para as duas grandezas de tempo do requisito R4. Na entrada digitada, um valor inválido deve ser recusado com uma mensagem clara e a pergunta repetida, em vez de derrubar o programa ou aceitar o

valor. As faixas válidas são as óbvias: ingresso não negativo, duração positiva, e seção crítica contida na duração da tarefa.

Um sorteio interessante não pode se perder

Cada sorteio produz um conjunto diferente, de modo que o cenário responsável por um resultado curioso desaparece ao fim da execução. O simulador deve permitir **gravar** o conjunto de tarefas em uso e **recarregá-lo** depois, de modo que qualquer cenário, digitado ou sorteado, possa ser reexaminado e discutido.

R3. Métricas

O simulador apresenta, por tarefa e em média:

Grandeza	Fórmula	Definição
Tempo de execução t_t	$t_{\text{conclusão}} - t_{\text{ingresso}}$	tempo de vida da tarefa, do ingresso à conclusão
Tempo de processamento t_p	dado de entrada	total de processador que a tarefa demanda
Tempo de espera t_w	$t_t - t_p$	tempo perdido na fila de tarefas prontas
Primeira execução	$t_{1^{\text{a}} \text{ exec.}} - t_{\text{ingresso}}$	tempo até a tarefa receber o processador pela primeira vez

A quarta linha não é o tempo de resposta

O tempo de resposta t_r mede o intervalo entre a chegada de um **evento** à máquina e o resultado imediato de seu processamento, como o intervalo entre apertar uma tecla e o caractere aparecer na tela. Mede-se um evento de cada vez, e não a tarefa inteira, de modo que este simulador não tem como calculá-lo. O tempo até a primeira execução é um **indicador de agilidade**, e é nele que o Round-Robin compensa o que perde em t_t e t_w .

A **justiça** tampouco tem expressão numérica: duas tarefas de comportamento e prioridade semelhantes devem ter durações de execução semelhantes. Um escalonador injusto apresenta boas médias e ainda assim mantém alguma tarefa indefinidamente sem execução. É o que o requisito R8 trata.

R4. Quantum e custo da troca de contexto

As duas grandezas de tempo do escalonador, o quantum t_q e o custo de uma troca de contexto t_{tc} , são **variáveis definidas por quem executa** o simulador, como o número de tarefas de R2. Nenhuma das duas pode estar fixada no código-fonte.

- t_{tc} vale para todos os algoritmos e pode ser zero, caso em que a troca é desprezada e o modelo se reduz ao das tabelas da Aula 5;
- t_q só faz sentido sob Round-Robin, e precisa ser **maior** que t_{tc} : se a troca consumisse a fatia inteira, nenhum trabalho útil seria realizado. Recusar essa combinação com uma mensagem clara faz parte do requisito.

A **eficiência** resume o efeito das duas:

$$E = \frac{t_q}{t_q + t_{tc}}$$

Em um processador real, t_q costuma ficar entre 1 e 100 ms. Com $t_q = 10$ ms, o escalonador é acionado ao menos 100 vezes por segundo em cada processador.

A eficiência só existe com quantum

E é propriedade da **configuração** do escalonador, não do conjunto de tarefas: dois cenários diferentes sob o mesmo t_q e o mesmo t_{tc} têm a mesma eficiência. Nos cinco algoritmos sem quantum ela não está definida, e o simulador deve indicar isso em vez de exibir um valor.

R5. Recursos de uso exclusivo e inversão de prioridades

Uma tarefa pode declarar uma **seção crítica**: o trecho de sua execução em que detém, com **exclusão mútua**, um recurso R . Sob PRIOp, a tarefa que solicita R enquanto ele está ocupado fica **suspensa**, sai do conjunto de tarefas prontas e não pode ser escolhida, por mais alta que seja sua prioridade.

O simulador deve permitir distinguir as duas situações:

- **bloqueio direto**: a tarefa espera por quem detém o recurso. É inevitável e tem duração limitada pelo tamanho da seção crítica;
- **inversão de prioridades**: a tarefa espera por terceiros, de prioridade intermediária, que sequer usam o recurso. É evitável e tem duração ilimitada.

R6. Herança de prioridade

Mecanismo de correção habilitável por parâmetro. Havendo tarefas suspensas à espera de R , a tarefa detentora assume a maior prioridade entre as delas e a sua própria, até liberar o recurso.

A herança é temporária

A prioridade herdada vale enquanto durar a espera e precisa ser revertida na liberação. Uma herança que não é desfeita transforma a tarefa de menor prioridade na de maior prioridade do sistema pelo resto da execução.

R7. Teto de prioridade

Segundo protocolo de correção, alternativo à herança. Cada recurso recebe, em tempo de projeto, um valor igual à maior prioridade entre as tarefas que podem usá-lo. Qualquer tarefa que obtenha o recurso assume esse valor **imediatamente**, sem esperar que um conflito ocorra.

Reativo contra preventivo

A herança age depois que a disputa aparece; o teto age na obtenção do recurso. A diferença de implementação é de uma linha: a condição para elevar a prioridade deixa de ser a existência de tarefas bloqueadas e passa a ser simplesmente a existência de um detentor. A diferença de comportamento não é pequena, e cabe ao simulador exibi-la.

R8. Envelhecimento

A prioridade efetiva de uma tarefa pronta cresce α unidades por unidade de tempo em que ela permanece na fila sem ser despachada, e volta ao valor base assim que recebe o processador. O valor de α é configurável.

O simulador precisa permitir demonstrar a eliminação da **inanição**: um cenário em que, sob prioridade cooperativa, uma tarefa de prioridade baixa espera indefinidamente, e em que o envelhecimento a faz ser atendida.

R9. Gerador de cenários

Geração **aleatória** de conjuntos de tarefas, com o número de tarefas escolhido por quem executa, conforme R2. O mesmo gerador serve a dois usos: sortear um cenário único para observar em detalhe, e sortear um **lote** de cenários para comparar os seis algoritmos do requisito R1 pelas médias de t_t e t_w .

Cada execução dá números diferentes

O sorteio é livre e não se repete, de modo que os valores absolutos das médias mudam de uma execução para outra. Não tente reproduzir os números da seção seguinte: o que precisa se manter é a **ordenação** entre os algoritmos. Com poucas amostras a ordenação oscila; com algumas dezenas ela se estabiliza, e é justamente essa estabilização que o requisito serve para mostrar.

O simulador deve ainda apresentar o **diagrama de tempo** dos cenários, no formato usado em sala: uma linha por tarefa, o tempo no eixo horizontal e os intervalos de execução destacados.

R10. Entrega executável

O projeto é entregue como um **programa que abre com dois cliques**. Não será montado ambiente, instalada dependência nem digitado comando para avaliar a entrega.

- o programa precisa funcionar **sem nenhum argumento** de linha de comando, porque um duplo clique não passa argumento algum;
- toda a interação de R2 e R4, quantidade de tarefas, dados ou sorteio, quantum e custo da troca, acontece **dentro** do programa depois de aberto;
- a janela não pode fechar sozinha ao terminar, nem diante de erro: uma mensagem que desaparece antes de ser lida equivale a nenhuma mensagem.

Vale qualquer forma que satisfaça isso: um executável gerado com PyInstaller, uma janela gráfica em **tkinter**, que acompanha a instalação padrão do Python, ou um arquivo de inicialização associado ao interpretador. Dependendo do menor número possível de bibliotecas externas: cada uma é uma chance a mais de o programa não abrir na máquina de quem avalia.

Teste em uma máquina limpa

Funcionar no computador em que foi escrito não é evidência de nada. Antes de entregar, abra o programa com dois cliques, a partir da pasta descompactada, sem o editor de código aberto e sem ativar nenhum ambiente. É exatamente assim que a entrega será avaliada.

3 Convenções obrigatórias

O comportamento interno é livre, mas os **números** não são. As dez convenções abaixo são as mesmas adotadas em sala e determinam os valores de referência da seção seguinte. Divergir de qualquer uma delas produz resultados diferentes dos esperados.

- C1. O tempo é discreto e avança de uma em uma unidade. Nos cenários das aulas a unidade é o segundo.
- C2. Prioridade: valor maior significa prioridade mais alta.
- C3. Desempate: menor instante de ingresso, depois menor identificador.
- C4. Ocorre troca de contexto sempre que a tarefa despachada é diferente da última que ocupou o processador, inclusive no primeiro despacho.

- C5. O custo da troca é **descontado** da fatia concedida, nunca somado a ela. Sob Round-Robin, a tarefa executa $t_q - t_{tc}$ unidades úteis quando houve troca.
- C6. A tarefa que esgota o quantum volta à cauda da fila **depois** das que ingressaram naquele mesmo instante.
- C7. A seção crítica é medida no tempo de execução **própria** da tarefa. Início 1 e duração 4 significa que a tarefa obtém o recurso após executar 1 unidade útil e o mantém até ter executado 5.
- C8. $t_w = t_t - t_p$. O tempo de espera engloba fila, suspensão no recurso e troca de contexto.
- C9. O teto de um recurso é calculado sobre as tarefas que **declaram** seção crítica nele, ainda que a disputa não chegue a ocorrer. Herança e teto são protocolos **alternativos** e não se combinam.
- C10. O envelhecimento conta o tempo decorrido desde o último despacho da tarefa, ou desde o seu ingresso caso ela ainda não tenha executado.

Quando o relógio fica ocioso

Se nenhuma tarefa ingressou ainda, o relógio **salta** para o próximo instante de ingresso. É um caso que passa despercebido enquanto o relógio está em zero e só aparece quando existe um intervalo ocioso no meio da simulação. Pelo menos um dos cenários de correção verifica exatamente isso.

4 Cenários de validação

Os quatro cenários abaixo são a verificação objetiva do projeto. Os valores apresentados são os corretos: reproduzi-los, com as convenções da seção anterior, é o que demonstra que o simulador funciona.

4.1 Cenário da Aula 5

	t_1	t_2	t_3	t_4	t_5
ingresso	0	0	1	3	5
t_p	5	2	4	1	2
prioridade	2	3	1	4	5

Alg.	T_t	T_w	1ª exec.	trocas
FCFS	8,0	5,2	5,2	5
RR ($q = 2$)	8,4	5,6	2,8	8
SJF	5,8	3,0	3,0	5
SRTF	5,4	2,6	2,4	6
PRIOc	6,6	3,8	3,8	5
PRIOp	5,6	2,8	2,2	7

Os seis valores de T_t e T_w são os mesmos apresentados em sala. Reproduzi-los é o primeiro sinal de que o simulador está correto.

4.2 Cenário da Aula 5 com custo de troca

As mesmas cinco tarefas, com $c = 1$ e $q = 4$.

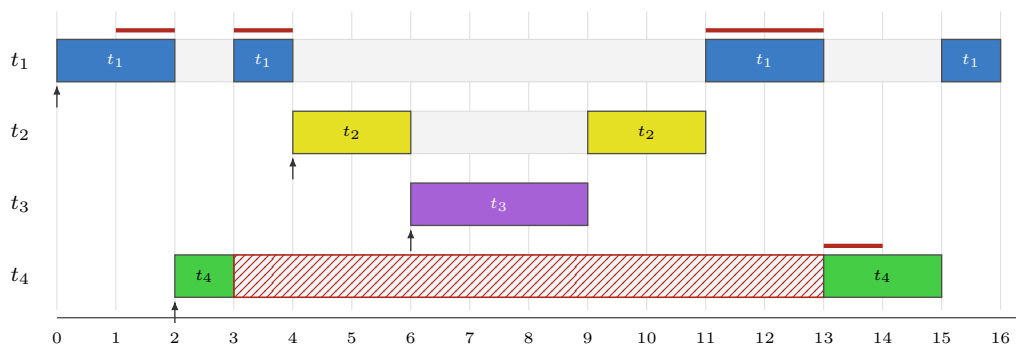
Algoritmo	T_t	T_w	E
FCFS	11,0	8,2	não definida
RR ($q = 4$)	13,4	10,6	$4/5 = 0,800$

O FCFS não tem quantum, logo não tem eficiência definida, ainda que as trocas de contexto atrasem todas as tarefas. Sob Round-Robin, cada fatia de 4 unidades entrega 3 de trabalho útil, e a eficiência da configuração é $4/(4 + 1)$.

4.3 Cenário da Aula 6: inversão de prioridades

Quatro tarefas sob PRIOp. Apenas t_1 , de menor prioridade, e t_4 , de maior prioridade, disputam o recurso R .

	t_1	t_2	t_3	t_4
ingresso	0	4	6	2
t_p	6	4	3	3
prioridade	1	2	3	4
seção crítica	[1, 5)	—	—	[1, 2)

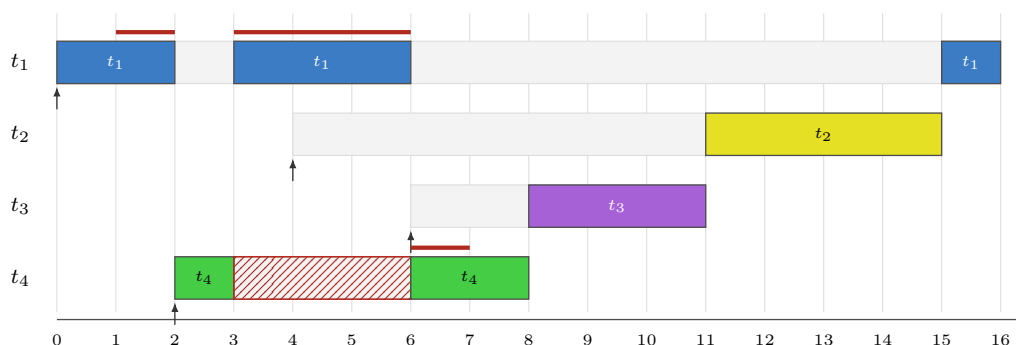


A área hachurada marca o bloqueio de t_4 . O traço superior marca a posse de R . Entre os instantes 4 e 11, t_2 e t_3 ocupam o processador enquanto t_4 , a tarefa mais prioritária do sistema, permanece bloqueada. Nenhuma das duas usa R .

	t_1	t_2	t_3	t_4		
conclusão	16	11	9	15	$T_t = 9,75$	$T_w = 5,75$
t_t	16	7	3	13		
t_w	10	3	0	10		

4.4 Cenário da Aula 6: herança de prioridade

As mesmas quatro tarefas, com o protocolo habilitado.



No instante 3, t_4 bloqueia e t_1 herda a prioridade 4. A partir daí t_2 e t_3 não conseguem mais preemptar o detentor do recurso, que conclui a seção crítica no instante 6 e devolve a prioridade 1.

	t_1	t_2	t_3	t_4		
conclusão	16	15	11	8	$T_t = 9,50$	$T_w = 5,50$
t_t	16	11	5	6		
t_w	10	7	2	3		

O bloqueio de t_4 cai de 10 para 3 unidades. A média geral melhora pouco, de 5,75 para 5,50, e essa desproporção é proposital: o protocolo não existe para melhorar médias, e sim para limitar o atraso da tarefa mais prioritária.

4.5 Cenário da Aula 6: teto de prioridade

O mesmo cenário, agora sob o protocolo de teto. O teto de R vale 4, a maior prioridade entre t_1 e t_4 .

	t_1	t_2	t_3	t_4		
conclusão	16	15	11	8	$T_t = 9,50$	$T_w = 9,50 - 4 = 5,50$
t_t	16	11	5	6		
t_w	10	7	2	3		

Sequência de execução: $t_1[0,5)$ $t_4[5,8)$ $t_3[8,11)$ $t_2[11,15)$ $t_1[15,16)$.

As médias coincidem com as da herança, mas o caminho é outro: t_1 já valia 4 desde o instante 1, de modo que t_4 sequer recebe o processador antes de R ser liberado. O bloqueio não foi encurtado de 3 para 0, foi **prevenido**. É a diferença entre um protocolo reativo e um preventivo, e o simulador precisa exibi-la.

O preço do teto aparece quando a disputa nunca chega a ocorrer. Com t_1 (ingresso 0, duração 6, prioridade 1, seção crítica $[1,5)$), t_2 (ingresso 2, duração 3, prioridade 2, sem recurso) e t_4 (ingresso 12, duração 2, prioridade 4, seção crítica $[0,1)$):

	Sequência de execução					t_w de t_2	T_w
herança	$t_1[0,2)$	$t_2[2,5)$	$t_1[5,9)$	$t_4[12,14)$		0	1,00
teto	$t_1[0,5)$	$t_2[5,8)$	$t_1[8,9)$	$t_4[12,14)$		3	2,00

t_4 só ingressa no instante 12, muito depois de R ter sido liberado, e portanto nunca disputa coisa alguma. Ainda assim o teto elevou t_1 durante toda a seção crítica e adiou t_2 em 3 unidades.

4.6 Inanição e envelhecimento

Seis tarefas sob prioridade cooperativa. A tarefa t_1 tem prioridade 1 e concorre com cinco tarefas de prioridade 5 que ingressam de duas em duas unidades de tempo.

	t_1	t_2	t_3	t_4	t_5	t_6
ingresso	0	0	2	4	6	8
t_p	4	2	2	2	2	2
prioridade	1	5	5	5	5	5

	Sequência de execução						t_w de t_1	T_w
PRIOc, sem envelhecimento	$t_2[0,2)$	$t_3[2,4)$	$t_4[4,6)$	$t_5[6,8)$	$t_6[8,10)$	$t_1[10,14)$	10	1,67
$\alpha = 1$	$t_2[0,2)$	$t_3[2,4)$	$t_1[4,8)$	$t_4[8,10)$	$t_5[10,12)$	$t_6[12,14)$	4	2,67
$\alpha = 2$	$t_2[0,2)$	$t_1[2,6)$	$t_3[6,8)$	$t_4[8,10)$	$t_5[10,12)$	$t_6[12,14)$	2	3,00

Sem envelhecimento, t_1 é atendida somente após todas as demais. Com $\alpha = 1$, sua prioridade efetiva cruza o valor 5 no instante 4 e ela passa à frente. Quanto maior α , mais cedo ocorre o cruzamento, e mais as tarefas prioritárias pagam por isso: T_w do conjunto sobe de 1,67 para 3,00. O envelhecimento não é um ganho, é uma troca.

4.7 Lote de cenários gerados

Médias sobre 50 cenários de cinco tarefas, ingressos até 8, durações até 6, prioridades até 5, quantum 2 e custo de troca nulo. Uma execução qualquer produziu:

Algoritmo	T_t	T_w	1ª exec.
FCFS	7,99	4,36	4,36
RR	9,39	5,76	2,03
SJF	7,02	3,39	3,39
SRTF	6,84	3,20	3,01
PRIOc	7,97	4,34	4,34
PRIOp	8,23	4,60	3,77

É aqui que as afirmações da Aula 5 deixam de ser propriedades de um cenário particular: o SRTF apresenta o menor T_w e o Round-Robin o menor tempo médio até a primeira execução na média de muitos conjuntos de tarefas, e não por acaso naquelas cinco tarefas específicas.

Estes números não devem ser reproduzidos

Cada execução sorteia outros cenários, e as médias mudam na primeira casa decimal. Esta tabela é apenas uma amostra do que esperar em ordem de grandeza. O único resultado que precisa se repetir é a **ordenação**: SRTF com o menor T_w e Round-Robin com o menor tempo até a primeira execução, em qualquer execução com algumas dezenas de amostras.